

and miss the market window. Assuming the designers make the circuit work, they have either a cost issue or manufacturing issue. Implementing a circuit through such ‘assembled’ approach can produce pretty nasty and unpredictable results.

Let us see why designers tend to take this route.

1. IC product lifecycles are shortening. The devices now have only 7-8 year life, necessitating continuous learning of new devices, which the current day designers are not able to cope up with.

2. Increase in operating speed with reduced silicon geometry. This calls for strong high-speed design skills and good basics, which are a rarity with the current crop of engineers.

3. Almost a similar situation exists in the embedded software. In fact, the situation in the embedded software is even worse as most embedded software developers are only integrating the software components. Programming in C and Assembly language has become a rare skill, causing a big problem. A proper software architecture should be the right solution.

We have seen product development cycles extending and fewer companies/startups succeeding. Above all, interest in electronic product based startups has started vaning due to the perceived issues.

Why this is critical

Every electronic product needs detailed specifications to perform better and work in the environment for which it has been designed. When products are in critical domains like medical devices, avionics, and automotive, they need documentation and a strict process adherence. In fact, most of these domains demand submission of documentation to the regulatory authorities, and these documents are typically design history files (DHF).

When you pick a canned design and use it in your product, the documentation is not available. In addition, most of these vendors put a disclaimer that the circuit provided is for demonstration purpose and they are indemnified when the designers

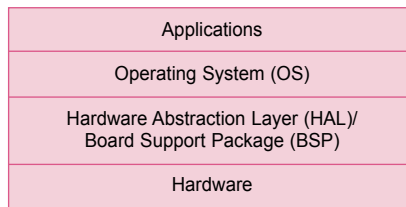


Fig. 1: Typical simplified software architecture of an OS based system

use it in their product design.

If the product to be designed has lots of communication functions, reference (canned) designs do not scale up. This is true for both hardware and software. Today this issue plagues the IoT designs as most IoT ‘Things’ and ‘Gateways’ are communication-intensive and a properly designed hardware and software is a must for proper working.

Some examples

Here are a couple of examples of how the designs that used the canned designs and associated software have performed. One of the IoT designs of a startup used a reference based hardware and code that was supplied with the reference board. But as the testing progressed the team faced following issues:

1. Response time of the system was slower than the requirement and team ended up revisiting the code and the hardware. What transpired was that the interrupt section was not handled correctly in the hardware and software leading to the slow response.

2. In the same product the target bill of material (BoM) and the actual BoM were completely off target. Primary reason was that the reference design used almost all the parts (digital and analogue) from the same vendor, while other than microcontroller the team got lower-cost parts with better performance from other vendors. This happens especially with analogue parts and certain standard digital devices.

3. One key aspect in designs where off-the-shelf (canned) code is used is the care that is needed in the design stage. For example, embedded software for actual system can be broadly classified into following types:

(a) **Bare metal coding.** This is typically applicable to systems that

use very few I/Os and are designed based on 8-bit processors. Bare metal code is specific to the hardware and offers less flexibility. But it is preferred for simpler systems that need to be produced in large volumes as such coding results in less memory consumption and tuned response time to meet the application needs.

(b) **OS based application.** This type of coding needs a basic operating system (like Free RTOS or Linux) and the application is written over the OS. This type of coding is good for high processor performance and multitasking application needs. This type of system typically uses 32-bit processors and is fairly complex. (The irony is that most designers use the canned design based on OS and walk into trouble.) The designers need to understand that OS will always be canned in both open source and vendor supplied software. But the section of the code that pertains to the hardware is the critical item in the overall design.

Typical simplified software architecture of an OS based system is shown in Fig. 1.

When designers choose a product architecture that uses an OS, they have to ensure that the OS is ported to the processor selected/used and the processor vendor gives the HAL/BSP for the OS. Most common mistake seen is that designers choose the processor, OS, and the BSP, but their design also has additional devices outside the controller for which the BSP or OS will not have support. Designers tend to forget this step and later struggle to make the board work.

Even messier problem noticed is the network stack (portion of the code which manages the networking functions in an OS). Other than a few versions of Linux, most OS and RTOS normally do not come with network stack, leading to huge issues. If a native networking stack is missing, porting a stack is going to be a challenge and can lead to performance, interoperability, and memory shortage issues. So, if a canned software is being planned, ensure the network stack is reliable